CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

# Objective 37: Tool choice and sequencing

> **Exam objective:** Configure the tool_choice parameter to guarantee tool invocation when structured output is required, and sequence multi-tool workflows so prerequisite data is obtained before dependent tools are called.

---

### Q1.

An application must always return a validated shipping quote object from a pricing tool before responding to users. Free-form model guesses are unacceptable. Which configuration best supports this requirement?

A. Leave tool choice automatic and rely on prompt wording alone B. Configure `tool_choice` to require the shipping quote tool when that workflow is invoked C. Disable tools so the model answers consistently D. Increase temperature to encourage tool exploration

**Answer:** B **Explanation:** When structured output from a specific tool is mandatory, `tool_choice` should enforce that invocation rather than leaving the decision optional. Prompting alone does not guarantee compliance. Disabling tools or raising temperature works against the requirement.

### Q2.

A workflow needs a customer ID from `search_customers` before it can call `get_contracts_for_customer`. What is the correct sequencing pattern?

A. Call `get_contracts_for_customer` first and hope the model infers the ID B. Call both tools in parallel every time C. First obtain the customer ID with `search_customers`, then call `get_contracts_for_customer` using the returned identifier D. Skip the search and fabricate a placeholder ID

**Answer:** C **Explanation:** The second tool depends on an identifier produced by the first, so the workflow must respect that dependency. Parallel execution is inappropriate when the downstream input is unknown. Fabricated IDs and inference guesses undermine correctness.

### Q3.

An architect says, "We should force every tool call with `tool_choice` so the model is predictable." What is the best response?

A. Correct, because forcing tools is always safer than allowing flexibility B. Incorrect, because `tool_choice` should be forced mainly when a specific tool or structured output is required; otherwise it can unnecessarily constrain the workflow C. Correct, because forced tools reduce authentication needs D. Incorrect, because `tool_choice` only affects formatting, not tool use

**Answer:** B **Explanation:** `tool_choice` is most valuable when there is a concrete need to guarantee invocation or constrain the model to a required tool path. Forcing all calls can cause unnecessary or inappropriate tool use. It does not remove authentication requirements.

### Q4.

You need a final answer that includes a normalized JSON record created by a tool, not a prose summary assembled by the model. Which design is best?

A. Ask the model nicely to mirror the schema in text B. Require the normalization tool through `tool_choice` and use its output as the authoritative structured result C. Provide several example JSON documents and disable tools D. Let the model choose whether a tool is worth calling

**Answer:** B **Explanation:** If a tool's structured output is the system of record, the application should guarantee the call rather than treating it as optional. Prompt examples help but do not enforce compliance. Automatic choice may still result in a free-form response.

### Q5.

A multi-step process uses `lookup_order`, `get_order_items`, and `issue_refund`. When is parallel execution appropriate?

A. Always, because faster is better B. Never, because tools must run one at a time C. Only after dependencies are known; for example, once `lookup_order` returns an order ID, independent item or policy lookups may proceed if they do not depend on each other D. Only if the model explicitly asks permission

**Answer:** C **Explanation:** Sequencing should follow data dependencies, not a blanket rule. Some steps must be serialized until required identifiers are available, after which truly independent branches may run in parallel. "Always" and "never" are both too rigid.

### Q6.

An agent must check whether a user exists before creating a new account to avoid duplicates. Which workflow ordering is best?

A. Create the account first, then search for duplicates later B. Force the create tool with `tool_choice` and skip the check C. Search or retrieve first, then conditionally call the create tool only if no matching account exists D. Call delete, then create, then search

**Answer:** C **Explanation:** This is a classic prerequisite check: existence must be established before a destructive or state-changing creation step occurs. Forcing creation too early defeats the safeguard. The other sequences create avoidable errors or data churn.

### Q7.

Why is relying solely on prompt text weaker than configuring `tool_choice` when a compliance workflow requires an audit logging tool to run?

A. Because prompt text cannot influence model behavior at all B. Because `tool_choice` provides a stronger control mechanism to guarantee the logging tool is invoked rather than merely suggested

C. Because `tool_choice` encrypts the audit logs D. Because prompts are only used in offline systems

**Answer:** B **Explanation:** Prompts can encourage behavior, but they do not provide the same level of enforcement as an explicit tool invocation requirement. In compliance-sensitive paths, guarantees matter. `tool_choice` addresses selection, not encryption.

### Q8.

A workflow requires `search_employee`, then `get_manager`, then `schedule_meeting_with_manager`. What design flaw exists if the application tries to schedule the meeting immediately after the search step?

A. None, because the manager is implied by the employee result B. The meeting step depends on manager information that has not yet been retrieved C. Scheduling should always happen before retrieval for responsiveness D. Search tools cannot be used in workflows with calendars

**Answer:** B **Explanation:** The downstream scheduling call requires information produced by the manager lookup. Skipping that prerequisite creates a missing-data dependency. The other options assume facts not established by the workflow.

### Q9.

An application uses `tool_choice=auto` in a path where the user sometimes asks for plain explanation and sometimes asks for a formal compliance record. What is the best design?

A. Keep one mode for all cases and hope the model distinguishes them B. Dynamically require the compliance-record tool only for the branch that mandates structured output, while allowing auto selection for explanation-only branches C. Force the compliance tool even for casual explanations D. Remove all tools and answer in prose

**Answer:** B **Explanation:** Different branches have different guarantees. Use forced invocation where structured output is mandatory and allow automatic selection where flexibility is acceptable. This preserves correctness without over-constraining unrelated requests.

### Q10.

What is the main risk of forcing a downstream tool before prerequisite data is collected?

A. Lower token usage B. The tool may be called with missing, guessed, or invalid inputs, producing unreliable results C. Faster response times D. Improved schema adherence

**Answer:** B **Explanation:** Forcing a tool too early can lead to calls that lack the identifiers or parameters required for correct execution. That produces unreliable outputs or failures. Any token or timing effects are secondary to the correctness risk.

### Q11.

A tool `resolve_ticket_by_id` produces a machine-readable closure object required by an external workflow engine. The model often wants to answer "Done" without calling the tool. What should you do?

A. Accept the prose response if it sounds confident B. Force `resolve_ticket_by_id` with `tool_choice` whenever closure must be recorded in the workflow engine C. Rename the tool to `done` D. Raise the model temperature

**Answer:** B **Explanation:** If an external system requires the tool's structured closure object, then the invocation must be guaranteed rather than optional. Confidence wording is not a substitute for actual workflow completion. Temperature and naming do not solve the guarantee problem.

### Q12.

An architect wants to optimize latency in a workflow with `get_user_profile` and `get_user_usage_history`, both of which require the same known user ID and neither depends on the other's result. What is the best sequence?

A. Run them in parallel after the user ID is available B. Always serialize them because tools should never overlap C. Run usage history first, then discover the user ID later D. Skip one tool and infer its data from the other

**Answer:** A **Explanation:** Once prerequisite data is present and the two calls are independent, parallelization is appropriate and reduces latency without sacrificing correctness. Serializing independent steps adds delay unnecessarily. Skipping required data sources invites incomplete results.

### Q13.

Your team says, "If a tool returns structured JSON, we do not need to think about ordering." Why is this incorrect?

A. Structured outputs eliminate all dependencies automatically B. A tool can still depend on identifiers, filters, or context that must be gathered by earlier steps before the structured call is valid C. JSON tools cannot be forced D. Ordering matters only for write operations

**Answer:** B **Explanation:** Output structure and workflow dependency are separate concerns. A tool may return perfect JSON yet still require upstream discovery to supply valid inputs. Ordering matters for both reads and writes whenever one step feeds another.

### Q14.

In a data-enrichment pipeline, `find_company` may return multiple matches. The next tool, `get_financials`, requires one canonical company ID. What is the best orchestration approach?

A. Immediately call `get_financials` for the first search result without inspection B. Add a disambiguation or selection step after `find_company`, then call `get_financials` with the confirmed ID C. Run `get_financials` on every candidate every time D. Skip `find_company` and invent an ID from the company name

**Answer:** B **Explanation:** When an upstream search can yield multiple candidates, the workflow must resolve ambiguity before passing input to a dependent tool. Blindly choosing the first result or inventing IDs reduces reliability. Calling every candidate may be wasteful and incorrect.

## Q15.

What is the best summary principle for combining `tool_choice` and sequencing in agent workflows?

A. Force every tool and ignore dependencies B. Use automatic choice everywhere and trust the model to improvise missing inputs C. Force tool invocation when a specific structured result is required, and order tools so each dependent call occurs only after its prerequisites are available D. Prefer prose answers over tool outputs because they are simpler

**Answer:** C **Explanation:** This principle captures both halves of the objective: guarantee essential tool use when needed, and respect data dependencies across multi-tool workflows. Over-forcing or over-trusting the model leads to avoidable failures. Structured workflows should be explicit about both control and order.